



Práctica 5

Organización y Arquitectura de Computadoras

Docente: Garcia Rocha Jose Isabel

Alumno:

- Chaparro Herrera Hugo Giovanni - 2200357

Tijuana, Baja California a 3 de octubre de 2025



Objetivo

Seleccionar el modo de direccionamiento adecuado, para manejo de memoria en aplicaciones de sistemas basados en microprocesador, mediante el análisis de los diversos modos de direccionamiento existentes, de forma responsable y eficiente.

Dificultades durante el desarrollo

Durante el desarrollo de la práctica, la única dificultad que se presentó fue la limitación de tiempo. Aunque logré completar cada uno de los puntos establecidos, al llegar al último no fue posible finalizarlo debido a la falta de tiempo disponible.

Desarrollo

1. (10%) Crear una subrutina que capture caracteres, los almacena en memoria y después mostrar la cadena con puts, recibe en edx la dirección de la cadena a utilizar, debe retornar la longitud de la cadena.

El programa inicia mostrando un mensaje al usuario para ingresar un mensaje que indica al usuario ingresar una cadena que termine con * para poder ser capturada.

```
; imprimir prompt
mov edx, msg
call puts
call salto

; esi = inicio
mov ebx, cad
mov edi, 0

capturar:
    call getche          ; se captura el caracter y se almacena en al
    cmp al, '*'          ; se compara con el indicador de fin de cadena
    je fin               ; si al == *, se sale del ciclo capturar

    cmp al, 13           ; si al == ENTER, no se captura
    je capturar

    cmp al, 10           ; si al == NUEVA_LINEA, no se captura
    je capturar

    mov [ebx + edi], al   ; se coloca el caracter capturado en ebx + indice (edi)
    inc edi              ; se incrementa el indice
    jmp capturar         ; se reinicia el ciclo

fin:
    call salto           ; se imprime un salto de linea
    mov edx, msg2        ; se manda el contenido a ebx con el prompt para poder ser impreso
    call puts            ; se imprime el prompt de cadena capturada
    call salto

    mov edx, cad
    call puts
    call salto
```



```
koria@Giolaptop:/mnt/c/Users/hugog/OneDrive/Documentos/6to Semestre/OAC/Practica5$ ./Practica5
Ingresa una cadena: (que termine en *)
cadena*
Cadena ingresada:
cadena
```

2. (20%) Crear una subrutina que muestra cadenas de caracteres en terminal, pueden utilizar cualquier símbolo especial para finalizar la cadena, recibe en `edx` la dirección de la cadena a imprimir, eso se debe modificar también en la subrutina anterior, finaliza la impresión utilizando el símbolo especial.

Primero se crea la subrutina, esta subrutina la llamaremos `newputs`, su comportamiento se basa en ir accediendo a cada posición de la cadena moviendo un índice y comparando si ya llegamos al indicador de cadena terminada, en este caso, usamos `'%'` como indicador. Para esto será necesario modificar un poco la captura de la cadena, ya que debemos colocar el terminador de la cadena una vez capturada.

```
; modificacion al capturar la cadena
mov ebx, cad           ; se manda la direccion de la cadena a ebx
mov al, '%'            ; se coloca el caracter '%' en al
mov [ebx + edi], al    ; se coloca el % al final de la cadena

; subrutina nueva para imprimir
newputs:
    pushad
    prnt:
        mov al, [edx + esi] ; se manda a al el caracter inicial
        cmp al, '%'        ; se compara al con %
        je finputs         ; si es verdadero, se sale del ciclo

        call putchar       ; se imprime al caracter actual
        inc esi            ; se incrementa el indice
        jmp prnt           ; se reinicia el ciclo de impresion
    finputs:
    popad
    ret

; codigo
; impresion del prompt para la cadena actualizada
mov edx, msg4
call puts
call salto

; imprime la cadena actualizada
mov edx, cad
call newputs
call salto
```



```
koria@GioLaptop:/mnt/c/Users/hugog/OneDrive/Documentos/6to Semestre/OAC/Practica5$ ./Practica5
Ingresa una cadena: (que termine en *)
cadena*
Cadena ingresada:
cadena
Cadena con %:
cadena%
Cadena impresa con newputs:
cadena
```



Código





```
%include "../Libreria/pc_io.inc"
; se incluye la libreria
section .text
    global _start:

_start:
    ; imprimir prompt
    mov edx, msg
    call puts
    call salto

    ; esi = inicio
    mov ebx, cad
    mov edi, 0

    capturar:
        call getche          ; se
        captura el caracter y se
        almacena el al
        cmp al, '*'          ; se
        compara con el indicador de fin
        de cadena
        je fin               ; si
        al == *, se sale del ciclo
        capturar

        cmp al, 13           ; si
        al == ENTER, no se captura
        je capturar

        cmp al, 10           ; si
        al == NUEVA_LINEA, no se captura
        je capturar

        mov [ebx + edi], al ; se
        coloca el caracter capturado en
        ebx + indice (edi)
        inc edi              ; se
        incrementa el indice
        jmp capturar         ; se
        reinicia el ciclo

    ; imprimir cadena capturada
fin:
    call salto              ; se
    imprime un salto de linea
    mov edx, msg2           ; se
    manda el contenido a ebx con el
    prompt para poder ser impreso
    call puts               ; se
    imprime el prompt de cadena
    capturada
    call salto
```



```
    mov edx, cad
    call puts
    call salto

    mov ebx, cad           ; se
manda la direccion de la cadena
a ebx
    mov al, '%'           ; se
coloca el caracter '%' en al
    mov [ebx + edi], al    ; se
coloca el % al final de la
cadena

    ; imprime el prompt de la
cadena con %
    mov edx, msg3
    call puts
    call salto

    ; se imprime la cadena
actualizada
    mov edx, cad
    call puts
    call salto

    ; impresion del prompt para
la cadena actualizada
    mov edx, msg4
    call puts
    call salto

    ; imprime la cadena
actualizada
    mov edx, cad
    call newputs
    call salto

    ; SYS_EXIT
    mov eax, 1
    mov ebx, 0
    int 80h

newputs:
    pushad
    prnt:
        mov al, [edx + esi]
    ; se manda a al el caracter
inicial
        cmp al, '%'
    ; se compara al con %
        je finputs
    ; si es verdadero, se sale del
ciclo
```

```
        call putchar
    ; se imprime al caracter actual
        inc esi
    ; se incrementa el indice
        jmp prnt
    ; se reinicia el ciclo de
impresion
    finputs:
    popad
    ret

salto:
    pushad
    mov al, 13
    call putchar

    mov al, 10
    call putchar
    popad
    ret

section data
    msg: db "Ingresa una cadena:
(que termine en *)", 0x0
    len: equ $-msg

    msg2: db "Cadena ingresada:
", 0x0
    len2: equ $-msg2

    msg3: db "Cadena con %:",
0x0
    len3: equ $-msg3

    msg4: db "Cadena impresa con
newputs:", 0x0
    len4: equ $-msg4

    pal: db "Si es palindromo.",
0x0
    len5: equ $-pal

    noPal: db "No es
palindromo.", 0x0
    len5: equ $-noPal

section .bss
    cad resb 256
```



Referencias Bibliográficas

- [02. NASMx86: partes del código fuente](#)
- [Guide to x86 Assembly](#)
- [Using as - Assembler Directives](#)
- [NASM Assembly Language Tutorials - asmtutor.com](#)
- [Assembly - System Calls](#)